WasmEdge LFX Mentorship (2026, Term 3) - Pre-test Report and Proposal

Support for the Wide Arithmetic Proposal (Issue #5153)

Basic Information

- Full Name: Ghule Vedant Babasaheb
- Email: vedant.ghule24@pccoepune.org
- GitHub: never954 - https://github.com/never954

Check List

- [x] I have read the WasmEdge Mentorship Program Policy (https://github.com/WasmEdge/WasmEdge/blob/master/docs/mentorship-policy.md)
- [x] I have read the LFX Mentorship Standards of Excellence (https://docs.linuxfoundation.org/lfx/mentorship/standards-of-excellence)

Pre-test 1

Output:

```
30
```

Pre-test 2

Approach

In `Executor::execute` at `lib/executor/engine/engine.cpp`, the per-instruction dispatch reads the current opcode and runs a large `switch`. I added a check at the top of that dispatch, before the `switch`, which fires for every SIMD (v128) instruction and logs its name and byte offset.

The opcodes are generated from a single table in `include/common/enum.inc`. Every SIMD opcode uses the `0xFD` prefix and they are listed in the table one after another. Because the `OpCode` enum values

follow the same order, the SIMD opcodes occupy one continuous numeric range. That means a single range check identifies any SIMD instruction, and I did not have to touch each `case` individually.

```
const OpCode Code = Instr.getOpCode();
if (Code >= OpCode::V128__load &&
    Code <= OpCode::I32x4__relaxed_dot_i8x16_i7x16_add_s) {
  spdlog::info("[SIMD] executing {} at offset {}"sv, Code, Instr.getOffset());
}
```

The instruction name is printed through the existing `fmt::formatter<OpCode>`.

Patch

```
diff --git a/lib/executor/engine/engine.cpp b/lib/executor/engine/engine.cpp
--- a/lib/executor/engine/engine.cpp
+++ b/lib/executor/engine/engine.cpp
@@ -103,6 +103,12 @@ Expect<void> Executor::execute(Runtime::StackManager &StackMgr,
 #endif
   -> Expect<void> {
     const AST::Instruction &Instr = *PC;
+    const OpCode Code = Instr.getOpCode();
+    if (Code >= OpCode::V128__load &&
+        Code <= OpCode::I32x4__relaxed_dot_i8x16_i7x16_add_s) {
+      spdlog::info("[SIMD] executing {} at offset {}"sv, Code,
+                   Instr.getOffset());
+    }
     switch (Instr.getOpCode()) {
     // Control instructions
     case OpCode::Unreachable:
```

WAT file

```
(module
  (func (export "run") (result i32)
    v128.const i32x4 1 2 3 4
    v128.const i32x4 10 20 30 40
    i32x4.add
    i32x4.extract_lane 0))
```

Output

Run with `wasmedge --reactor simd.wasm run`:

```
[info] [SIMD] executing v128.const at offset 33
[info] [SIMD] executing v128.const at offset 51
[info] [SIMD] executing i32x4.add at offset 69
```

```
[info] [SIMD] executing i32x4.extract_lane at offset 72
11
```

Pre-test 3

Approach

A small C++ program that uses the LLVM API to build a function `i128 add128(i128, i128)` using `IRBuilder`, JIT-compiles it with `MCJIT`, then calls it with two 128-bit inputs and prints the result.

Implementation

```cpp
#include "llvm/ADT/APInt.h"
#include "llvm/ADT/SmallString.h"
#include "llvm/ExecutionEngine/ExecutionEngine.h"
#include "llvm/ExecutionEngine/MCJIT.h"
#include "llvm/IR/IRBuilder.h"
#include "llvm/IR/LLVMContext.h"
#include "llvm/IR/Module.h"
#include "llvm/IR/Verifier.h"
#include "llvm/Support/TargetSelect.h"
#include "llvm/Support/raw_ostream.h"
#include <cstdint>
#include <memory>
#include <string>

using namespace llvm;

static __int128 apintToI128(const APInt &V) {
  uint64_t Lo = V.getRawData()[0];
  uint64_t Hi = V.getRawData()[1];
  return (__int128)(((unsigned __int128)Hi << 64) | Lo);
}
static APInt i128ToAPInt(__int128 X) {
  unsigned __int128 U = (unsigned __int128)X;
  uint64_t Words[2] = {(uint64_t)U, (uint64_t)(U >> 64)};
  return APInt(128, ArrayRef<uint64_t>(Words, 2));
}

int main(int argc, char **argv) {
  StringRef AStr = (argc > 1) ? argv[1] : "170141183460469231731687303715884105727";
  StringRef BStr = (argc > 2) ? argv[2] : "1";
  APInt A(128, AStr, 10), B(128, BStr, 10);

  InitializeNativeTarget();
```

```
  InitializeNativeTargetAsmPrinter();

  auto Ctx = std::make_unique<LLVMContext>();
  auto Mod = std::make_unique<Module>("wide_arith", *Ctx);
  IRBuilder<> Builder(*Ctx);

  Type *I128 = Builder.getInt128Ty();
  FunctionType *FT = FunctionType::get(I128, {I128, I128}, false);
  Function *F = Function::Create(FT, Function::ExternalLinkage, "add128", Mod.get());

  BasicBlock *Entry = BasicBlock::Create(*Ctx, "entry", F);
  Builder.SetInsertPoint(Entry);
  auto Args = F->arg_begin();
  Value *ArgA = &*Args++;
  Value *ArgB = &*Args;
  ArgA->setName("a"); ArgB->setName("b");
  Value *Sum = Builder.CreateAdd(ArgA, ArgB, "sum");
  Builder.CreateRet(Sum);

  if (verifyFunction(*F, &errs())) { errs() << "verify failed\n"; return 1; }
  outs() << "=== Generated LLVM IR ===\n";
  Mod->print(outs(), nullptr);

  std::string ErrStr;
  ExecutionEngine *EE = EngineBuilder(std::move(Mod))
                            .setErrorStr(&ErrStr)
                            .setEngineKind(EngineKind::JIT)
                            .create();
  if (!EE) { errs() << "EE: " << ErrStr << "\n"; return 1; }
  EE->finalizeObject();

  auto Add128 = (__int128 (*)(__int128, __int128))EE->getFunctionAddress("add128");
  __int128 Result = Add128(apintToI128(A), apintToI128(B));

  SmallString<64> AS, BS, RS;
  A.toString(AS, 10, false);
  B.toString(BS, 10, false);
  i128ToAPInt(Result).toString(RS, 10, false);
  outs() << "=== Result ===\n" << AS << " + " << BS << " = " << RS << "\n";
  delete EE;
  return 0;
}
```

Build command

```
clang++ $(llvm-config --cxxflags) llvm_i128.cpp \
  $(llvm-config --ldflags --libs core executionengine mcjit native --system-libs) \
  -o llvm_i128
```

Output

With default inputs (`INT128_MAX + 1`):

```
=== Generated LLVM IR ===
define i128 @add128(i128 %a, i128 %b) {
entry:
  %sum = add i128 %a, %b
  ret i128 %sum
}
=== Result ===
170141183460469231731687303715884105727 + 1 = 170141183460469231731687303715884105728
```

Proposal

Summary

The goal is to implement the four WebAssembly Wide Arithmetic instructions `i64.add128` , `i64.sub128` , `i64.mul_wide_s` , and `i64.mul_wide_u` in WasmEdge, in all three execution modes (interpreter, LLVM AOT, and JIT). The work touches opcode registration, the loader, the validator, the interpreter, and the LLVM compiler, and needs to pass the upstream spec tests.

Problem statement

WebAssembly integers only go up to 64 bits. Programs that need 128-bit arithmetic, such as big-integer libraries and cryptography code, currently build it out of many 64-bit instructions with an explicit compare and branch to detect carry. That is slower and larger than it needs to be, because 64-bit CPUs already do 128-bit add and subtract in hardware using the carry flag and `adc` . The Wide Arithmetic proposal adds four instructions so a runtime can map them directly onto that hardware path. The biggest win happens in the LLVM AOT/JIT compilation, where `i64.add128` compiles to a native `add` plus `adc` pair.

Instruction semantics

Instruction	Opcode	Stack type	Meaning
`i64.add128`	`0xFC 19`	`[i64 i64 i64 i64] -> [i64 i64]`	128-bit add
`i64.sub128`	`0xFC 20`	`[i64 i64 i64 i64] -> [i64 i64]`	128-bit subtract (LHS – RHS)
`i64.mul_wide_s`	`0xFC 21`	`[i64 i64] -> [i64 i64]`	signed 64×64 → 128-bit product
`i64.mul_wide_u`	`0xFC 22`	`[i64 i64] -> [i64 i64]`	unsigned 64×64 → 128-bit product

The spec defines the stack top-to-bottom for `add128` and `sub128` as `RHS_high, RHS_low, LHS_high, LHS_low`, so `v1 = LHS`, `v2 = RHS`, and `sub128` computes `v1 − v2`. The `0xFC` opcode block in WasmEdge currently ends at sub-opcode 17 (`table.fill`), so 19 to 22 are unused.

Implementation plan

Opcode registration and proposal gating

In `include/common/enum.inc` I add four opcode entries and a new `WideArithmetic` proposal. WasmEdge enables proposals by default and exposes `--disable-<proposal>` flags, so gating means adding `WideArithmetic` to the default set, wiring up the disable flag, and adding `hasProposal()` checks where needed. The loader's decoder is generated from `enum.inc`, so registering the opcodes there also makes them decodable.

Loader

In `lib/loader/ast/instruction.cpp` the four instructions carry no immediate operands, so they just join the no-immediate group. The mirror change goes in `lib/loader/serialize/serial_instruction.cpp` for the encode side, with round-trip tests under `test/loader`.

Validator

In `lib/validator/formchecker.cpp` the validator uses a generic `StackTrans(Take, Put)`. Since WasmEdge already ships Multi-value support, declaring the two-result signatures is enough:

- `add128` and `sub128` get `StackTrans({I64,I64,I64,I64}, {I64,I64})`
- `mul_wide_s` and `mul_wide_u` get `StackTrans({I64,I64}, {I64,I64})`

Interpreter

In `lib/executor/engine/engine.cpp`, next to the existing `I64__add` case, I add four new cases. The math uses `WasmEdge::uint128_t` and `int128_t` from `include/common/int128.h`. That header already handles the platform split: `__int128` on GCC and Clang, and a hand-written struct on MSVC. So one implementation works on all platforms and the Windows build does not need special intrinsics.

LLVM AOT and JIT

In `lib/llvm/compiler/numericInstr.cpp`, next to the `I64__add` lowering, I add the four cases. For `add128` and `sub128` I zero-extend the four `i64` values to `i128`, put them back together into the two operands, do `createAdd` or `createSub`, then split the `i128` result into two `i64`s using `trunc` for the low half and `lshr 64` plus `trunc` for the high half. For `mul_wide_u` and `mul_wide_s` I `zext` or `sext` both inputs to `i128`, do `createMul`, and split. Pushing two results is two `stackPush` calls, since the AOT stack is a plain vector. LLVM handles turning this into native `add`/`adc` and widening multiply sequences.

Current progress

I have a working prototype of the interpreter path on a local `master` checkout. It covers stages 1 to 4 above (opcodes, loader parse, validator, and the four interpreter handlers) and comes to about 60 lines across four files.

The `add128` handler is the core of it:

```
case OpCode::I64__add128: {
  const uint64_t BHi = StackMgr.pop().get<uint64_t>();
  const uint64_t BLo = StackMgr.pop().get<uint64_t>();
  const uint64_t AHi = StackMgr.pop().get<uint64_t>();
  const uint64_t ALo = StackMgr.pop().get<uint64_t>();
  const uint128_t Res = ((static_cast<uint128_t>(AHi) << 64) | ALo) +
                        ((static_cast<uint128_t>(BHi) << 64) | BLo);
  StackMgr.push(ValVariant(static_cast<uint64_t>(Res)));
  StackMgr.push(ValVariant(static_cast<uint64_t>(Res >> 64)));
  return {};
}
```

It pops the four halves, rebuilds the two 128-bit values, adds them with a single `+` on `uint128_t`, and pushes the low and high halves of the result. The single `+` is where the carry is handled: the compiler turns it into a native `add`/`adc`, so there is no manual compare-and-branch. `sub128` is the same with `-`. `mul_wide_u` and `mul_wide_s` pop two values, zero- or sign-extend them to `uint128_t`/`int128_t`, multiply once, and push both halves.

A `.wat` module using all four instructions parses, validates, and runs. I checked the results against hand computation:

Function	Inputs	Result (lo, hi)
`add128`	$(2^{64} - 1) + 1$	`0, 1`
`sub128`	$2^{64} - 1$	`-1, 0`
`mul_wide_u`	$(2^{64} - 1)^2$	`1, -2`
`mul_wide_s`	$(-1) \cdot (-1)$	`1, 0`

Results are printed as signed `i64`, so `-1` is `0xFFFFFFFFFFFFFFFF` and `-2` is `0xFFFFFFFFFFFFFFFE`. This gave me the operand order for `sub128` and the test vectors I plan to reuse for AOT/JIT differential testing.

Testing

I plan to run the same test vectors through the interpreter, AOT, and JIT and check they all agree. That covers zero, wrap around the 64-bit boundary, signed vs unsigned multiply, and `INT64_MIN` and `INT64_MAX` edges.

For the upstream spec tests, `test/spec/CMakeLists.txt` fetches the suite from `WasmEdge/wasmedge-spectest` at pin `ee503c07`, and the `WASMEDGE_SPECTEST_FOLDERS` list does not include `wide-arithmetic` yet. The upstream `WebAssembly/wide-arithmetic` repo ships `test/core/wide-arithmetic.wast` with tests for all four instructions. So making the spec suite pass depends on those tests landing in the spectest mirror and bumping the pin. I would like to check with the mentors in Week 1 whether the mentee sends the PR to add the folder and bump the pin, or if that mirror is updated on a separate cadence, so I can plan Week 10 accordingly.

I will also add loader round-trip tests under `test/loader` for the new opcodes.

Timeline

Weeks	Milestone
1	Confirm operand order against the spec. Check spec-test mirror plan with mentors. Add opcodes, `WideArithmetic` proposal, and configuration gating.
2	Loader parse and serialize; round-trip tests.
3	Validator signatures and validation tests.
4	Finalize interpreter handlers from the prototype, add edge-case tests.
5	Interpreter hardening. Extra time in case anything slips.
6-7	LLVM lowering for `add128` and `sub128` on AOT and JIT.
8-9	LLVM lowering for `mul_wide_s` and `mul_wide_u`. Differential tests across interpreter, AOT, and JIT.
10	Wire up the spec tests and get them passing.
11	Cross-platform check on MSVC via `int128.h`. Review round. Extra time.
12	Docs in `WasmEdge/docs`. Final review.

Risks

The spec test dependency is the main external risk, which is why I want to raise it in Week 1. Getting the operand order or the lo/hi halves the wrong way round is easy to do; I plan to keep the spec open when writing each handler and rely on the differential tests. LLVM API differences across versions can also bite, so I will build against the same LLVM version CI uses and stick to the basic `IRBuilder` calls (`zext`, `sext`, `shl`, `lshr`, `trunc`, `add`, `sub`, `mul`).

Availability

I have university mid-semester and end-semester exams from late November to early December. That is roughly 15 to 20 days where I will not be able to put in a full week. The timeline puts the interpreter and validator work in the first half, and leaves the exam period lining up with documentation and review, so the heavy implementation weeks are not affected. I will share the exact dates with the mentors at the start of the mentorship.